



FOR SOFTWARE ENGINEERS: THE NIGHT BEFORE VERSION

Modern Java

Everything that actually shipped

Sixteen features, each compiled on Java 8, 11, 17, 21, 25 and 26. The compile errors on the older releases are the evidence.

You have not written a switch expression by hand in a long time. The agent writes it, it compiles, the tests pass, and you approve it. That is fine, right up until someone asks you to explain sealed interfaces with no editor in front of you.

This sheet is the ladder: what arrived when, what it replaced, and the one table nobody else prints, which is what has not arrived at all.

Every version and availability claim on this sheet is stated as at September 2026 and was checked against the OpenJDK JEP index on the day it was built. A feature that was preview then may be final by the time you read this. The dates are the point of the page, not decoration.

In scope

- The sixteen features that are final, and the release each became final in
- What each one replaced, in one line
- Two measurements taken on one machine: virtual threads, and compact object headers
- The features people believe shipped and have not
- The one that was withdrawn entirely
- The answer that beats listing version numbers

Out of scope

- Anything still in preview, except in the table that says so
- Framework behaviour. This is the platform, not Spring
- Build tooling, which is a different sheet
- Any claim that a newer release is automatically the right one for your team

Read the ladder first. Everything else on this page is a consequence of which release your production runtime is actually on.

01 The ladder, checked September 2026

Java 10	var, local variable type inference
Java 11	HttpClient, final
Java 14	Switch expressions
Java 15	Text blocks
Java 16	Records, and pattern matching for instanceof
Java 17	Sealed classes
Java 21 (LTS)	Virtual threads, sequenced collections, record patterns
Java 22	Foreign Function and Memory API
Java 24	Stream gatherers
Java 25 (LTS)	Scoped values, compact source files, module imports, flexible constructor bodies
Java 26	HTTP/3 in the existing HttpClient
Compact object headers	Opt in from 25, default from 27

The left column is the release a feature became FINAL, not the release it first appeared in preview. That distinction is most of what an interviewer is testing.

03 Virtual threads, measured rather than asserted

Ten thousand tasks, each waiting a tenth of a second, which is what almost every backend thread is actually doing. On a pool of 200 platform threads: 5,226ms, because everyone queues behind an expensive OS thread.

Change one word to a virtual thread executor. Same blocking code, same machine: 152ms. Thirty four times.

THE PART AN INTERVIEWER ACTUALLY CHECKS: a virtual thread is not a faster thread. It is a thread that UNMOUNTS. When it blocks, the JVM parks it and hands the carrier thread to somebody else. Nothing got faster; the waiting stopped costing a thread.

Which is also the limit. Work that is CPU bound rather than blocked gains nothing, because there was never a carrier thread sitting idle to reclaim.

Virtual threads are final in Java 21.

JEP 444, Virtual Threads

10,000 tasks each sleeping 100ms: 5,226ms on a 200 thread platform pool, 152ms on virtual threads, same machine, same source.

Measured on the machine that built this sheet, not quoted

04 Records, patterns, and what a record is NOT

A constructor, two getters, equals, hashCode and toString, for a class holding an order id and a quantity: fourteen lines, and every one of them is a place to make a mistake. Forget a field in equals and you have a bug that survives review, because it looks like typing.

THE INTERVIEW QUESTION IS NEVER WHAT A RECORD IS. It is what a record is not. It is final, so you cannot extend it. Its fields are final. And equals is derived from the components, which is exactly why it is the wrong shape for a JPA entity with a database identity.

Pattern matching for instanceof removes the cast you already proved was safe. Sealed types plus record patterns remove the default branch, because the compiler knows the list of subtypes is closed and can prove the switch exhaustive.

The payoff is not brevity. Add a fourth payment event tomorrow and every switch that needed updating stops compiling. The compiler finds the call sites instead of production finding them.

Records are final in Java 16 and pattern matching for instanceof is final in Java 16.

JEP 395, Records

Sealed classes are final in Java 17 and record patterns in Java 21.

JEP 409, Sealed Classes

05 The missing break, which does not throw

Four order statuses, an old style switch, and one missing break on SHIPPED. Run it: SHIPPED prints "cancelled". Not an exception. Not a warning. A quietly wrong answer that sits in production until a customer phones about an order they were told was cancelled.

A switch expression returns a value rather than assigning to one, so there is nothing to break out of, and it is exhaustive over the enum. Add a fifth status tomorrow and it stops compiling.

This is the feature an interviewer is most likely to open with, and it is the one that most often gets an answer about syntax when the answer is about failure modes.

Switch expressions are final in Java 14.

JEP 361, Switch Expressions

A missing break in an old style switch prints the wrong status rather than raising an error.

The compiler itself, on the release named

06 var and text blocks, the two that are argued about most and matter least

var is not dynamic typing, and this is settled by running it rather than debating it: the runtime class is identical, and asking the variable for a method it does not have still fails AT COMPILE TIME. var changes what you type, not what the compiler knows.

The honest rule: use it when the right hand side already says the type. Avoid it when the type is the only documentation the line has.

Text blocks are the smallest feature here with the biggest daily payoff. The part that makes them safe rather than merely pretty is that the compiler strips the common leading whitespace, so the indentation of your Java file does not leak into your JSON.

var is local variable type inference, final in Java 10. The inferred type is static.

JEP 286, Local-Variable Type Inference

Text blocks are final in Java 15, and incidental leading whitespace is removed by the compiler.

JEP 378, Text Blocks

07 Sequenced collections and stream gatherers

Before Java 21 there was no way to ask a `LinkedHashSet` for its last element. It is an ordered collection, it knows the order, and you still had to iterate the whole thing. On a `List` you had `get(size() - 1)`, an off by one waiting to happen. Two different answers to the same question.

Java 21 gave every ordered collection `getFirst`, `getLast` and `reversed`. A small interface closing a gap open since the first collections framework.

Streams could always map, filter and reduce, and could never look at more than one element at a time, so a rolling window meant leaving the stream for an index loop or a custom `Splitter`.

Gatherers, final in Java 24, put windowing back inside the pipeline.

Sequenced collections are final in Java 21.

JEP 431, Sequenced Collections

Stream gatherers are final in Java 24.

JEP 485, Stream Gatherers

08 The `ThreadLocal` leak, fixed structurally

A production incident in five lines. Set a user on a thread from a pool. That task finishes. A completely different task runs on the same pooled thread and asks who the user is. It says `alice`.

That is one request seeing another request's identity, and `ThreadLocal` will do it every time, because nothing ever told it when the value stopped being valid.

Scoped values fix the `STRUCTURE` rather than the `API`. The value is bound for the duration of one call and is unbound when that call returns, so there is no state left on the thread for the next task to find.

It also matters more now than it did, because virtual threads mean far more threads and far more reuse.

Scoped values are final in Java 25.

JEP 506, Scoped Values



09 The ceremony that finally went

COMPACT SOURCE FILES: no class, no public static void main, no String array of arguments you were never going to read. Write the method, run the file. MODULE IMPORTS replace the screen of imports with import module java.base.

Why this matters more than it looks: the distance between having an idea and testing it in Java has been embarrassing for twenty years compared with almost every other language. That gap is now mostly closed.

FLEXIBLE CONSTRUCTOR BODIES end an argument you have had in a code review. Until Java 25 the super call had to be the first statement, so validating an argument meant either smuggling it into an expression or letting the parent constructor run on a value you already knew was invalid. Statements are now allowed before super, as long as they do not touch the object being constructed. The object never half exists.

Compact source files and instance main methods are final in Java 25.

JEP 512, Compact Source Files and Instance Main Methods

Module import declarations are final in Java 25.

JEP 511, Module Import Declarations

Flexible constructor bodies are final in Java 25.

JEP 513, Flexible Constructor Bodies

10 Three upgrades you get without changing code

THE HTTP CLIENT. In Java 8 a GET meant `HttpURLConnection`, a buffered reader, a manual loop and a finally block. Java 11 made `HttpClient` final: a builder, a request, a response, four lines. At the time of writing, Java 26 gave that same client HTTP/3. Same API, same four lines, QUIC underneath. The code you already wrote is the code that gets the new protocol.

FOREIGN FUNCTION AND MEMORY. Calling C used to mean a native shim compiled per platform, a `.so` and a `.dylib` shipped next to the jar, and a mistake in that shim taking the JVM down with it. It is now about six lines of plain Java with an arena that owns the memory deterministically. JNI is over and almost nobody noticed.

COMPACT OBJECT HEADERS. Object headers went from twelve bytes to eight. That sounds like nothing until you remember what most business applications are: a heap full of small objects. Five million cached order lines measured at 115 MB with default headers and 76 MB with compact headers on, 24 bytes per object down to 16. One flag, no code change.

`HttpClient` is final in Java 11; HTTP/3 support arrived in Java 26.

JEP 517, HTTP/3 for the HTTP Client API

The Foreign Function and Memory API is final in Java 22.

JEP 454, Foreign Function and Memory API

Five million cached order lines: 115 MB with default object headers, 76 MB with compact object headers. Opt in from Java 25, default from Java 27.

Measured on the machine that built this sheet, not quoted



11 What has NOT shipped, and this is the page that matters

Knowing this is the fastest way to sound like someone who has used the platform rather than read about it, and the compiler will say it for you: use `StructuredTaskScope` with no flags and you get "error: `StructuredTaskScope` is a preview API and is disabled by default".

STRUCTURED CONCURRENCY: still preview, and at the time of writing it is on its SEVENTH preview.

PRIMITIVE TYPES IN PATTERNS: still preview, fifth round.

LAZY CONSTANTS, formerly Stable Values: still preview.

VECTOR API: still an INCUBATOR module after twelve rounds.

AND ONE THAT WAS WITHDRAWN ENTIRELY: string templates reached a second preview and were then pulled. Any page that shows them as usable is wrong.

Structured concurrency is on its seventh preview, primitive types in patterns on its fifth, lazy constants still preview, and the Vector API still incubating after twelve rounds, as at September 2026.

OpenJDK JEP index, checked September 2026

String templates reached a second preview and were withdrawn.

OpenJDK JEP index, checked September 2026

error: `StructuredTaskScope` is a preview API and is disabled by default. (use `--enable-preview` to enable preview APIs)

The compiler itself, on the release named

12 The answer that beats listing versions

Asked what is new in Java, the weak answer is a list of version numbers, because it is checkable, forgettable, and available to anyone with a browser.

The stronger answer names the DIRECTION and one consequence of it: the platform spent a decade removing ceremony and is now removing the reasons you reached outside it. Virtual threads removed the reason to adopt reactive. FFM removed the reason to write JNI. Records and sealed types removed the reason to hand write a visitor.

Then pick one and say what it cost. Virtual threads gain nothing on CPU bound work. Records are the wrong shape for an entity with a database identity. An answer with a cost in it is the one that sounds like experience.

Naming a trade-off rather than a version distinguishes someone who has operated the platform from someone who has read the release notes.

Interview practice, not vendor documentation

Before you claim a feature in an interview

1. Which release made it FINAL, not which release previewed it?
2. What did it replace, in one sentence?
3. What does it cost, and where does it gain nothing?
4. Is it actually shipped, or is it on its seventh preview?
5. Is the runtime in my production actually on that release?
6. If I am quoting a number, did I measure it or read it?
7. For records: can I say what a record is NOT?
8. For virtual threads: can I say what unmounting means?
9. For var: can I say why it is not dynamic typing?
10. Can I name one thing I would deliberately not adopt yet, and why?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of these you have actually shipped.

Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates. Eclipse Temurin is a trademark of the Eclipse Foundation. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-14 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.